EMILLY DE BARROS DE OLIVEIRA
GIOVANNA PAULIN LEBEDIEFF DOS SANTOS
LUCAS LANA VIANNA
RODRIGO YUJI NONAKA RODRIGUES

RELATÓRIO TÉCNICO

MyKeeper

Equipe 4

Curitiba

2026

SUMÁRIO

1. Contextualização 3
2. Descrição da Solução 4
3. Padrões Utilizados 6
4. Análise de Aplicabilidade dos Padrões 7
5. Reflexões 8

1. Contextualização

O mercado de aplicativos e sistemas voltados à digitalização e organização de hábitos, incluindo a gestão alimentícia doméstica, vem crescendo de forma consistente, uma vez que famílias e indivíduos enfrentam dificuldades para controlar os alimentos que possuem em casa, definir o que devem comprar, lembrar de produtos prestes a vencer e decidir o que fazer com os alimentos disponíveis. Essas informações costumam ser registradas de forma manual ou em aplicativos separados, o que gera duplicadas e dificuldade em manter o estoque atualizado.

O My keeper é um sistema de gestão de estoque de alimentos, voltado a usuários domésticos, que unifica o cadastro de produtos, o estoque, as receitas, as listas de compras, as metas de consumo e o histórico de preços em um mesmo ambiente. A solução se justifica pela necessidade de uma ferramenta centralizada, simples e prática, que traga organização unificada, evite compras duplicadas, apoie o planejamento das compras e permita acompanhar a variação de preços com suporte ao usuário.

2. Descrição da Solução

O MyKeeper é uma aplicação em Java 21 com JavaFX para a gestão doméstica de alimentos, compras e suporte ao usuário. A partir de um menu principal, o usuário acessa dez módulos de cadastro (CRUD), com dados persistidos localmente em arquivos ".dat". Dessa maneira é possível gerenciar todos estes módulos, utilizando dos seguintes Padrões de Projeto para a sua realização e otimização de processos: Template Method, Observer, State e Strategy.

Principais funcionalidades

- **Cadastros básicos:** produtos (nome, categoria, unidade de medida), categorias (descrição e setor) e fornecedores (contato, CEP e chave PIX).
- **Estoque com ciclo de vida:** cada estoque segue o fluxo *Ativo* → *Inativo* → *Arquivado*. O estoque inativo pode ser reativado, só pode ser excluído quando inativo, e o arquivado fica para somente leitura.
- **Listas de compras com ciclo de vida:** fluxo "*Em elaboração*" → "*Finalizada*" → "*Comprada*". A lista finalizada pode ser reaberta, e a comprada fica bloqueada para edição.
- **Receitas:** cadastro com ingredientes, modo de preparo e tempo de preparo.
- **Metas de consumo:** limite mensal por categoria e unidade de medida.
- **Histórico de preços com alertas:** ao registrar um preço, o sistema avisa automaticamente aumentos acima de 10% e novos menores preços do produto. Todos os alertas são gravados em log de auditoria (alertas_preço.log).
- **Suporte ao usuário:** cadastro de contas de suporte e tickets com prioridade e fluxo *Aberto* → *Em andamento* → *Resolvido* → *Fechado*, com reabertura. O ticket só pode ser excluído enquanto aberto, e tickets resolvidos ou fechados não podem ser editados.
- **Ordenação configurável:** em todas as listagens, o usuário escolhe o critério de ordenação (ID, nome, categoria, data, preço, tempo de preparo, limite etc.).
- **Regras de negócio consistentes:** as restrições de status valem tanto na interface quanto na camada de dados, o que impede alterações indevidas por qualquer caminho.

3. Padrões Utilizados

Padrão	Problema identificado
Template Method	Dez repositórios e dez telas com o mesmo algoritmo copiado
State	Status como texto livre, sem regras de transição, edição ou exclusão
Strategy	Nenhum critério de ordenação; adicionar critérios exigiria if/switch em cada tela.
Observer	Variação de preço não era percebida; acoplar alertas ao cadastro seria rígido

3.1. Template Method

Classificação de intenção	**Operações:** como as operações e algoritmos são executados ou encapsulados dentro dos objetos
Problema identificado	As classes ArquivoXxx (10 cópias - ArquivoEstoque, Fonecedor, Produto, etc.) repetiam ler, salvar, gerar ID, adicionar, atualizar e deletar; As telas XxxUI (10 cópias - EstoqueUI, FornecedorUI, ProdutoUI, etc.) repetiam montagem, seleção na tabela, tratamento dos botões e mensagens. Só mudavam o tipo, o nome do arquivo e os campos.
Justificativa	A sequência de passos é a mesma em todos os módulos e o que varia é pequeno e bem delimitado. A herança foi escolhida por ser mais simples que a composição, pois cada tela e repositório é um caso particular do mesmo fluxo.
Classes envolvidas	**Repositórios:** RepositorioArquivo (AbstractClass), RepositorioComEstado, ArquivoProduto, ArquivoHistoricoPreco, ArquivoTicketSuporte e demais (ConcreteClass); Identificavel. **Telas:** CrudUI (AbstractClass), CrudComEstadoUI, ProdutoUI, TicketSuporteUI, HistoricoPrecosUI, etc.
Modo de aplicação	O padrão foi aplicado nas classes de repositórios (dados) e nas

	telas (interfaces). Foram criadas duas classes base que guardam o "passo a passo" que se repetia nos 10 módulos: - RepositorioArquivo (dados): validar, ler o arquivo, alterar a lista e gravar. Essa ordem não pode ser mudada pelas classes filhas. - CrudUI (tela): fixa o fluxo de montar a tela, tratar seleção na tabela e reagir aos botões (define o que acontece a cada clique). Cada módulo só informa o que é específico dele (nome do arquivo, campos do formulário)
Benefícios	Cada repositório simples passou a ter cerca de 10 linhas (só o caminho do arquivo) e cada tela só declara seus campos. Corrigir um bug do fluxo ou criar um novo módulo passa a ser uma alteração em um único lugar.

Evidência no código:

- model/RepositorioArquivo.java

  Passo abstrato (caminhoArquivo) e ganchos com implementação vazia

```
40        /** Passo obrigatório: nome do arquivo .dat da entidade. */
41        protected abstract String caminhoArquivo();
42
43        /** Gancho: valida uma entidade antes de ser adicionada. Padrão: não faz nada. */
44        protected void validar(T novo) { }
45
46        /** Gancho: valida a troca de {@code atual} por {@code novo}. Padrão: não faz nada. */
47        protected void validarAtualizacao(T atual, T novo) { }
48
49        /** Gancho: valida a remoção da entidade. Padrão: não faz nada. */
50        protected void validarRemocao(T atual) { }
51
52        /** Gancho: executado depois que a entidade foi gravada. Padrão: não faz nada. */
53        protected void aposAdicionar(T adicionado) { }
```

```java
    public final void adicionar(T novo) {
        validar(novo);                          // 1. gancho
        ArrayList<T> lista = lerLista();        // 2. passo fixo
        lista.add(novo);                        // 3. passo fixo
        salvarLista(lista);                     // 4. passo fixo
        aposAdicionar(novo);                    // 5. gancho
    }
```

- model/ArquivoProduto.java

  Subclasse só informa o caminho do arquivo

```java
/** Repositório de Produto (Template Method: só define o caminho do arquivo). */
public class ArquivoProduto extends RepositorioArquivo<Produto> {

    public static final ArquivoProduto INSTANCE = new ArquivoProduto();

    private ArquivoProduto() { }

    @Override
    protected String caminhoArquivo() {
        return "produtos.dat";
    }
}
```

- ui/CrudUI.java

  Fluxo fixo da tela ao adicionar

```
162     private void adicionar() {
163         try {
164             T novo = lerFormulario(repositorio().gerarId());
165             repositorio().adicionar(novo);
166             mensagem(nomeEntidade() + ": registro adicionado.");
167             limpar();
168             carregarTabela();
169         } catch (RuntimeException ex) {
170             mensagem(ex.getMessage());
171         }
172     }
```

3.2. State

Classificação de intenção	**Operações:** como as operações e algoritmos são executados ou encapsulados dentro dos objetos
Problema identificado	Ticket, lista de compras e estoque guardavam o status como texto livre. Nada impedia um ticket de ir de Aberto direto para Fechado, ser excluído em atendimento ou ser editado depois de resolvido. Cada regra teria de virar um if/switch espalhado pelas telas. - Ticket de suporte: aberto, em andamento, resolvido, fechado; - Lista de compras: em elaboração, finalizada, comprada; - Estoque: ativo, inativo, arquivado.
Justificativa	O que um registro pode fazer (ser editado, ser excluído, ir para o próximo passo) depende do estado em que ele está e muda conforme ele avança. Como o fluxo é pequeno e estável e cada estado tem regras próprias, faz sentido colocar cada regra dentro da classe do seu estado, em vez de espalhar condicionais por texto de status pelas telas. Esse é o cenário típico do State.
Classes envolvidas	**Context:** TicketSuporte, ListaCompras e Estoque (pela interface ComEstado). **State:** EstadoStatus. **ConcreteState:** TicketAberto, TicketEmAndamento, TicketResolvido, TicketFechado, ListaEmElaboracao, ListaFinalizada, ListaComprada, EstoqueAtivo, EstoqueInativo, EstoqueArquivado. **Apoio:** EstadosTicket/Lista/Estoque, RepositorioComEstado e CrudComEstadoUI.
Modo de aplicação	O padrão foi aplicado nas entidades que têm status. Cada estado virou uma classe (ConcreteState) que informa proximo(), anterior(), permiteEdicao() e permiteExclusao(). O nome do status continua sendo gravado como texto (compatível com os .dat antigos). A tela habilita ou desabilita botões consultando o estado; o repositório rejeita edições, exclusões e transições inválidas com IllegalStateException, de modo que a regra vale mesmo sem a interface.
Benefícios	As regras ficam junto de cada estado, sem condicionais por texto

	de status. Criar um novo estado (como Cancelado) é criar uma classe e ajustar a transição vizinha. A mesma estrutura atende as três entidades.

Evidências no Código:

- model/estado/EstadoStatus.java

  Interface do estado.

```java
public interface EstadoStatus {

    /** Nome persistido do estado (ex.: "Aberto"). */
    String getNome();

    /** Próximo estado do fluxo; retorna {@code this} se for um estado final. */
    EstadoStatus proximo();

    /** Estado anterior (reabertura); retorna {@code this} se não houver retorno. */
    default EstadoStatus anterior() { return this; }

    /** Texto do botão que executa a transição para o próximo estado. */
    String rotuloAvancar();

    default String rotuloVoltar() { return "Voltar status"; }

    default boolean permiteAvancar() { return proximo() != this; }

    default boolean permiteVoltar() { return anterior() != this; }

    default boolean permiteEdicao() { return true; }

    default boolean permiteExclusao() { return true; }
}
```

- model/estado/TicketEmAndamento.java

  Estado concreto.

```java
/** ConcreteState: ticket em atendimento. Pode ser editado, mas não excluído. */
public class TicketEmAndamento implements EstadoStatus {
    @Override public String getNome() { return "Em andamento"; }
    @Override public EstadoStatus proximo() { return new TicketResolvido(); }
    @Override public EstadoStatus anterior() { return new TicketAberto(); }
    @Override public String rotuloAvancar() { return "Marcar como resolvido"; }
    @Override public String rotuloVoltar() { return "Devolver para a fila"; }
    @Override public boolean permiteExclusao() { return false; }
}
```

- model/TicketSuporte.java

  Contexto delegando a transição ao estado.

```java
    @Override
    public void avancarStatus() {
        estado = getEstado().proximo();   // delegação ao objeto de estado
        status = estado.getNome();
    }
```

- model/RepositorioComEstado.java

  Regras de edição, exclusão e transição no repositório.

```java
    @Override
    protected void validarAtualizacao(T atual, T novo) {
        String nomeAtual = atual.getEstado().getNome();
        String nomeNovo = novo.getStatus();
        if (nomeAtual.equalsIgnoreCase(nomeNovo)) {
            // mesmo estado: é uma edição de dados
            if (!atual.getEstado().permiteEdicao()) {
                throw new IllegalStateException("Registro no estado \"" + nomeAtual + "\" não pode ser editado.");
            }
            return;
        }
        // estado diferente: só são aceitas as transições previstas pelo próprio estado
        boolean avancou = atual.getEstado().permiteAvancar()
                && atual.getEstado().proximo().getNome().equalsIgnoreCase(nomeNovo);
        boolean voltou = atual.getEstado().permiteVoltar()
                && atual.getEstado().anterior().getNome().equalsIgnoreCase(nomeNovo);
        if (!avancou && !voltou) {
            throw new IllegalStateException("Transição inválida de \"" + nomeAtual + "\" para \"" + nomeNovo + "\".");
        }
    }

    @Override
    protected void validarRemocao(T atual) {
        if (!atual.getEstado().permiteExclusao()) {
            throw new IllegalStateException("Registro no estado \"" + atual.getEstado().getNome() + "\" não pode ser excluído.");
        }
    }
```

3.3. Strategy

Classificação de intenção	**Operações:** como as operações e algoritmos são executados ou encapsulados dentro dos objetos
Problema identificado	As listagens apareciam sempre na ordem de gravação no arquivo. Oferecer vários critérios de ordenação com if/switch em cada tela repetiria a mesma lógica nas 10 telas e exigiria alterar todas a cada novo critério.
Justificativa	Existe uma família de algoritmos (os critérios de ordenação) com a mesma forma de uso: recebem uma lista e devolvem a lista ordenada. Quem escolhe qual usar é o usuário, durante a execução. Trocar o critério sem condicionais é exatamente o que o Strategy faz.
Classes envolvidas	**Strategy:** EstrategiaOrdenacao. **Base:** OrdenacaoPorComparador. **ConcreteStrategy:** OrdenacaoPorId, OrdenacaoPorTexto, OrdenacaoPorNumero, OrdenacaoPorData. **Context:** CrudUI.
Modo de aplicação	Cada critério de ordenação virou uma classe que implementa a interface EstrategiaOrdenacao, com dois métodos: getNome() (texto do seletor) e ordenar(lista) (devolve uma cópia ordenada). Uma classe base (OrdenacaoPorComparador) faz o trabalho de copiar e ordenar a lista, e cada estratégia concreta (por ID, texto, número ou data) só informa como comparar dois itens. Cada tela informa, pelo método estrategiasOrdenacao(), quais critérios oferece. A CrudUI preenche o seletor “Ordenar por” com eles e, quando o usuário escolhe um, o método carregarTabela() lê os dados do repositório e os exibe já ordenados pela estratégia escolhida. A tela nunca precisa saber como cada ordenação funciona.
Benefícios	Novos critérios entram como novas classes (ou novas instâncias parametrizadas) sem tocar nas telas. A lógica de ordenação é testável isoladamente, sem JavaFX.

Evidências no código:

- ordenacao/EstrategiaOrdenacao.java

  Interface da estratégia.

```java
public interface EstrategiaOrdenacao<T> {

    /** Texto exibido na lista de critérios da tela. */
    String getNome();

    /** Devolve uma nova lista ordenada (a original não é alterada). */
    List<T> ordenar(List<T> itens);
}
```

- ordenacao/OrdenacaoPorComparador.java

  Base comum: copia e ordena, com comparador() abstrato.

```java
public abstract class OrdenacaoPorComparador<T> implements EstrategiaOrdenacao<T> {

    private final String nome;

    protected OrdenacaoPorComparador(String nome) {
        this.nome = nome;
    }

    /** Critério de comparação da estratégia concreta. */
    protected abstract Comparator<T> comparador();

    @Override
    public List<T> ordenar(List<T> itens) {
        List<T> copia = new ArrayList<>(itens);
        copia.sort(comparador());
        return copia;
    }
```

- ui/CrudUI.java
  Contexto aplica a estratégia escolhida

```
212         /** Recarrega a tabela aplicando a estratégia de ordenação escolhida. */
213         protected final void carregarTabela() {
214             ArrayList<T> lista = repositorio().lerLista();
215             EstrategiaOrdenacao<T> estrategia = cmbOrdenacao.getValue();
216             dados.setAll(estrategia == null ? lista : estrategia.ordenar(lista));
217         }
```

- ui/ProdutoUI.java

  Critérios que a tela oferece.

```
47          @Override
48          protected List<EstrategiaOrdenacao<Produto>> estrategiasOrdenacao() {
49              return List.of(
50                      new OrdenacaoPorId<>(true), new OrdenacaoPorId<>(false),
51                      new OrdenacaoPorTexto<Produto>("Nome", Produto::getNome, true),
52                      new OrdenacaoPorTexto<Produto>("Nome", Produto::getNome, false),
53                      new OrdenacaoPorTexto<Produto>("Categoria", Produto::getCategoria, true));
54          }
```

3.4. Observer

Classificação de intenção	**Responsabilidade:** como as responsabilidades são distribuídas entre os objetos.
Problema identificado	O sistema já tinha um histórico de preços, cada vez que o usuário comprava um produto, o preço era gravado, mas não era possível perceber aumentos ou reduções. Colocar os alertas dentro do cadastro faria o repositório depender de cada tipo de aviso e da tela, e obrigaria a alterá-lo sempre que surgisse um alerta novo.
Justificativa	O cadastro de preços só precisa registrar o valor, mas várias partes do sistema querem reagir a esse registro: a tela mostra um aviso, e um arquivo guarda o histórico dos alertas. Essa lista de interessados muda (a tela só escuta enquanto está aberta, e novos alertas podem surgir). Por isso, o cadastro apenas avisa que “um preço foi registrado”, e cada interessado decide o que fazer. Isso é o Observer: avisar vários interessados sem que quem avisa precise conhecê-los (cenário publicação/assinatura).
Classes envolvidas	**Subject:** MonitorPrecos. **Observer:** ObservadorPreco. **ConcreteObserver:** AlertaAumentoPreco, AlertaMenorPreco, RegistroAlertasLog. **Evento:** VariacaoPreco. **Publicador:** ArquivoHistoricoPreco (gancho aposAdicionar do Template Method). **Assinantes:** HistoricoPrecosUI e Main.
Modo de aplicação	Ao adicionar um preço, ArquivoHistoricoPreco calcula o preço anterior e o menor preço já registrado do produto e chama MonitorPrecos.notificar(variacao) que repassa o evento a todos os observadores cadastrados. Cada observador decide se o evento lhe interessa: um avisa aumentos acima de 10%, outro avisa novos menores preços e outro grava um registro de auditoria em arquivo. A tela de histórico registra os dois alertas ao abrir e os remove ao fechar, e a Main registra o log durante toda a execução. Uma falha em um observador não impede os demais de serem avisados.
Benefícios	O repositório ArquivoHistoricoPreco (que grava os preços) não precisa conhecer os alertas. Para criar um alerta novo, basta

	cadastrá-lo, sem alterar o que já existe, e a falha de um alerta não afeta os demais.

Evidências no código:

- observer/ObservadorPreco.java

  Interface do observador.

```java
// Observer: quem quiser ser avisado de preços novos implementa e se registra no MonitorPrecos
public interface ObservadorPreco {
    void precoRegistrado(VariacaoPreco variacao);
}
```

- observer/MonitorPrecos.java

  Registrar e remover (assinar e cancelar).

```java
    public void registrar(ObservadorPreco o) {
        if (o != null && !observadores.contains(o)) observadores.add(o);
    }

    public void remover(ObservadorPreco o) {
        observadores.remove(o);
    }
```

  Notificar, com try/catch por observador.

```java
    public void notificar(VariacaoPreco variacao) {
        for (ObservadorPreco o : observadores) {
            try {
                o.precoRegistrado(variacao);
            } catch (RuntimeException e) {
                // se um observador der erro, os outros continuam sendo avisados
                System.err.println("Falha em observador de preço: " + e.getMessage());
            }
        }
    }
```

- model/ArquivoHistoricoPreco.java

  Publicação do evento no gancho aposAdicionar.

```java
@Override
protected void aposAdicionar(HistoricoPreco novo) {
    HistoricoPreco anterior = null;
    Double menor = null;
    for (HistoricoPreco h : lerLista()) {
        if (h.getId() == novo.getId() || h.getIdProduto() != novo.getIdProduto()) continue;
        if (menor == null || h.getPreco() < menor) menor = h.getPreco();
        boolean maisRecente = anterior == null
                || h.getData().isAfter(anterior.getData())
                || (h.getData().isEqual(anterior.getData()) && h.getId() > anterior.getId());
        if (!h.getData().isAfter(novo.getData()) && maisRecente) anterior = h;
    }
    MonitorPrecos.INSTANCE.notificar(new VariacaoPreco(
            novo.getIdProduto(),
            anterior == null ? null : anterior.getPreco(),
            menor, novo.getPreco(), novo.getData()));
}
```

- ui/HistoricoPrecosUI.java

  Assinatura ao abrir e cancelamento ao fechar.

```java
public HistoricoPrecosUI() {
    txtData.setPromptText("Data");
    montar();
    MonitorPrecos.INSTANCE.registrar(alertaAumento);
    MonitorPrecos.INSTANCE.registrar(alertaMenorPreco);
    setOnHidden(e -> {
        MonitorPrecos.INSTANCE.remover(alertaAumento);
        MonitorPrecos.INSTANCE.remover(alertaMenorPreco);
    });
}
```

- Main.java

  Registro do log durante toda a execução.

```
15          @Override
16          public void start(Stage stage) {
17              stage.setTitle("MyKeeper");
18
19              // o log de preços fica registrado enquanto o app estiver aberto
20              MonitorPrecos.INSTANCE.registrar(new RegistroAlertasLog());
```

4. Análise de Aplicabilidade dos Padrões

Padrão	**Tipo**	**Motivo por não se aplicar**
Singleton	Criacional	Tecnicamente foi aplicado (ex.: ArquivoProduto, ArquivoCategoria), mas não foi pensado para o uso como Singleton, pois não era um problema de design próprio, por isso não foi contado como padrão avaliado.
Factory Method	Criacional	Como cada tela é fixa, não há necessidade de decidir dinamicamente qual classe criar.
Abstract Factory	Criacional	O sistema não tem famílias de objetos que precisem trocar de kit.
Builder	Criacional	As entidades são simples, com construtores diretos. Não há objetos com muitos campos opcionais que justifiquem o uso do padrão.
Prototype	Criacional	Nenhum objeto é copiado, cada item é criado do zero a partir dos dados digitados.
Adapter	Estrutural	Não há sistema externo nem biblioteca incompatível para integrar, os dados são gravados direto em arquivos com serialização do Java.
Bridge	Estrutural	Não há caso para separar abstração e implementação para evoluírem separadamente.
Composite	Estrutural	Não há hierarquia de pastas ou subcategorias. Ex.: Produto guarda o nome da categoria como texto, não como um atributo que possa conter subcategorias.
Decorator	Estrutural	Não há necessidade de acrescentar funções extras a objetos em tempo de execução.
Facade	Estrutural	Não há um subsistema complexo que precise ser escondido.

Flyweight	Estrutural	O sistema lida com poucos objetos, não há milhares de itens iguais que pesem na memória para justificar o uso do padrão.
Proxy	Estrutural	Não há objetos que precisem de controle de acesso de outro objeto.
Chain of Responsibility	Comportamental	Não há pedidos que passem por uma sequência de responsáveis, cada ação é direta.
Command	Comportamental	Não há situação que precise transformar um pedido em um objeto independente. Ex.: há poucos botões e eles não tem comandos que se repitam em grande quantidade.
Interpreter	Comportamental	O sistema não tem linguagem própria ou expressões a interpretar.
Iterator	Comportamental	É utilizado somente listas e elas são percorridas utilizando somente o for, não há necessidade de implementar esse padrão.
Mediator	Comportamental	Não há muitos objetos conversando entre si que precisem de um intermediário. Ex.: As telas são independentes.
Memento	Comportamental	Não há métodos de restauração. Ex.: O HistoricoPreco guarda os preços ao longo do tempo, mas como registro, e não mecanismo de restauração.
Visitor	Comportamental	Não há algorítmos complexos que necessitem serem separados dos objetos nos quais operam.

5. Reflexões

A implementação dos padrões parecia uma tarefa fácil no começo, entretanto após o começo da execução, percebemos o verdadeiro trabalho e dificuldade, e como isso está ligado à maturidade envolvendo o assunto. Os padrões realmente resolvem problemas, porém nosso sistema era pequeno demais para muitas das soluções engenhosas, sendo um fator de dificuldade. O conhecimento sobre os padrões também resultou em um fator de dificuldade, já que não foi simples implementá-los de primeira. No fim, conseguimos implementar aquilo que nos propomos a implementar, e não chegamos a considerar mudanças de design, mas a experiência mostrou que não dá para simplesmente colocar padrões em qualquer sistema, e que faz-se necessária uma análise de contexto para ver se realmente vale a pena para o tamanho do sistema e para aquele contexto.